

Низькорівневе програмування та програмування мікроконтролерів

З використанням мови програмування **Assembler**
Stack

класрум **im3ozqq4**

Що таке стек?

Notes

Стек — це структура даних типу LIFO (last in, first out), яка підтримується апаратно у процесорах i386.

Він використовується для тимчасового збереження даних, параметрів виклику функцій та адрес повернення.

Стек є центральним механізмом у програмуванні на низькому рівні:
він дозволяє:

- викликати функції,
- передавати параметри,
- зберігати тимчасові дані
- забезпечувати рекурсивні виклики.

Синтаксис

```
mov ax, 5  
push ax
```

```
rsp : 0x4fffffffffed0
```

```
rsp : 0x4fffffffffece
```

Синтаксис

```
mov ax, 5  
push ax
```

```
pop bx
```

```
rsp : 0x4fffffffffed0
```

```
rsp : 0x4fffffffffece
```

```
rsp : 0x4fffffffffed0
```

У архітектурі i386 стек росте
вниз у пам'яті: кожна операція
PUSH зменшує ESP,
POP збільшує ESP

Інструкція PUSH зменшує ESP на розмір операнда та записує дане значення у пам'ять за новою адресою ESP.

Інструкція ROP читає дані
з вершини стека у вказаний
регістр або пам'ять,
після чого збільшує ESP

Можна безпосередньо класти значення

```
push 0x12
```

```
rsp : 0x4fffffffffed0
```

```
rsp : 0x4fffffffffec8
```

Але розмір даних відповідає розміру платформи (size, isize)

Розмір має значення

```
push 0x12345678  
push 0x09abcdef
```

```
pop ax  
pop rdx
```

Розмір має значення

```
push 0x12345678  
push 0x09abcdef
```

```
pop ax  
pop rdx
```

ef cd ab 09 00 00 00 00

78 56 34 12 00 00 00 00

ax : 0xcdef

rdx : 0x56780000000009ab

Баланс стеку

Операції **push** / **pop** мають відповідати одна одній інакше дані будуть пошкоджені

Баланс стеку

Операції **push** / **pop** зазвичай використовуються:

- у вкладених циклах, коли потрібно зберігати лічильник
- передачі параметрів при виклику підпрограм (next)

Зберегти все!

```
pusha
```

```
push eax  
push ecx  
push edx  
push ebx  
push esp  
push ebp  
push esi  
push edi
```

Відповідно є команда

пора

Відповідно порядок діставанням зі стеку дзеркальний

Можливо зберегти всі флаги

```
pushf
```

Зворотні операція

popf

Якщо потрібно просто “прибрати” значення зі стеку

```
push 0x12345678  
add rsp, 1  
  
pop ax
```

78 56 34 12 00 00 00 00

Якщо потрібно просто “прибрати” значення зі стеку

```
push 0x12345678  
add rsp, 1  
  
pop ax
```

78 56 34 12 00 00 00 00

ax : 0x3456

pusha + pushf / popa + popf

Використовуються коли потрібно “швидко” вставити блок іншого коду і не думати що має бути збережено

Приклад

вкладені цикли (сума матриці)

```
table db 1, 2, 3, 4  
      db 5, 6, 7, 8  
      db 9, 10, 11, 12
```

Використаємо можливості компілятора

```
table      db 1, 2, 3, 4
n_cols     = $ - table
|         |         |
|         |         | db 5, 6, 7, 8
|         |         | db 9,10,11,12
n_rows     = ($ - table) / n_cols
```

Підготовчі операції

```
xor    eax, eax  
mov    esi, table
```

Цикл по першому рядку

```
mov  cx, n_cols  
col1:  
    add  al, [esi]  
    inc  esi  
    loop col1
```

Цикл по колонкам

```
mov cx, n_rows  
row:  
    ...  
loop row
```

Комбінуємо цикли

```
mov    cx, n_rows
row:
  push  cx
  mov   cx, n_cols
col:
  add   al, [esi]
  inc   esi
  loop  col

  pop   cx
  loop  row
```

Збираємо
все
разом

```
xor    eax, eax
mov    esi, table

mov    cx, n_rows

row:
  push  cx
  mov   cx, n_cols

  col:
    add  al, [esi]
    inc  esi
    loop col

  pop   cx
  loop  row
```

Розмір стеку

Розмір стеку визначається не процесором, а операційною системою.

- У Linux типовий розмір стеку для потоку становить близько 8 МБ
- у Windows за замовчуванням резервується 1 МБ.
- Ці параметри можна змінювати через налаштування середовища або директиви компоновника.
- На голому залізі обмежено наявною пам'яттю.

Notes

Notes

Notes

Notes